

Searching ten million tickets, and handing out each one exactly once

Search 10,000,000 six-digit lottery tickets by a wildcard pattern, and distribute matches to concurrent users so that **no ticket ever reaches two people**. Every quantitative claim below is a measurement taken from a working implementation under enforced resource limits.

01 METADATA

Document control

TITLE	Lottery Search System — Design Document
AUTHOR	Natthawat Narin, Senior Backend Developer
STATUS	Final — submitted for review
VERSION	1.2
DATE	2026-09-05
REVIEWERS	—
REPOSITORY	

github.com/nthw-dev/challenge-lottery-search-system — this document, the decision records, the results analysis and every evidence bundle behind the figures

PROOF OF CONCEPT

github.com/nthw-dev/poc-lottery-search-system — the implementation and load tests that produced every measurement cited here

RESPONDS TO

`challenge-lottery-search-system.md`

DECISION LOG

`adr/` — eight architecture decision records, indexed in Appendix A

Every quantitative claim in this document is a measurement, taken from a proof of concept load-tested on Kubernetes under enforced resource limits. Section 8 indexes the evidence; Section 9 lists what remains unproven.

02 CONTEXT AND SCOPE

The real search space is one million, not ten million

The challenge: search 10,000,000 six-digit lottery tickets by a six-character pattern with `*` wildcards, and distribute matching tickets to concurrent users so that no ticket reaches two people. The deliverables are a solution architecture, a production storage recommendation with justification, a performance analysis, and a concurrency strategy that prevents duplicate distribution. Section 4 answers each directly; Sections 5 to 8 supply the detail and the evidence.

2.1 The premise

A six-digit ticket has $10^6 = 1,000,000$ **possible values**; ten million tickets therefore average ten tickets per number. The problem is not "search ten million rows" but "decide which of one million *numbers* match, then fetch their tickets". The second search space is ten times smaller, fixed regardless of ticket count, and small enough to stay in cache. Every decision below follows from this.

Result-set size varies by six orders of magnitude with the number of wildcards, so performance is reported per wildcard class (Section 6.2).

PATTERN	WILDCARDS	NUMBERS MATCHED	TICKETS MATCHED
123456	0	1	~10
123***	3	1,000	~10,000
****23	4	10,000	~100,000
3*	5	100,000	~1,000,000
*****	6	1,000,000	10,000,000

A wildcard may appear in any position, so an ordinary B-tree on the ticket number serves only patterns with fixed leading digits; `****23` cannot use it. That constraint is the technical core of the challenge.

03 GOALS AND NON-GOALS

Four testable goals, all met; six items deliberately excluded

3.1 Goals

Each goal was stated as a testable hypothesis before the proof of concept was built.

GOAL	TARGET	MEASURED	RESULT
G1 Wildcard search in any position at 10M rows on a 500m-CPU / 128Mi API pod	p95 < 100 ms for every wildcard class at 100 concurrent users	p95 46.7 ms, flat across classes, 6,068 req/s	met (Section 6.2)
G2 Concurrent users searching the same pattern never receive the same ticket	zero duplicates under 200-way contention	342,412 allocations, 0 duplicates, 0 ledger/state mismatches	met (Section 8.1)

GOAL	TARGET	MEASURED	RESULT
G3 The chosen index strategy beats a plain LIKE baseline at acceptable storage cost	$\geq 10\times$ on the hardest pattern; index size comparable to alternatives	four orders of magnitude on the exact pattern; 201 MB of indexes against 405 MB for the per-digit alternative	met (Section 6.2)
G4 Stability under sustained mixed load within the pod's memory limit	no OOM kill, memory well under 128Mi over a 10-minute soak	942,015 requests, 0 errors, 0 restarts, peak 26 MiB	met (Section 7.3)

3.2 Non-goals

Out of scope, so that the design proves one database sufficient before anything is added to it:

- Authentication and authorization
- Payment flow, prize drawing, and draw-period scheduling
- Horizontal scaling of the database, replication, read replicas
- Any caching layer; Redis is excluded deliberately
- A production observability stack (Prometheus, Grafana, tracing)
- CI/CD pipeline

04 DESIGN SUMMARY

Four deliverables, four decisions, each justified by a measurement

Answers to the four deliverables the challenge asks for. Each decision is justified by a measurement taken on the proof of concept and expanded in the sections named.

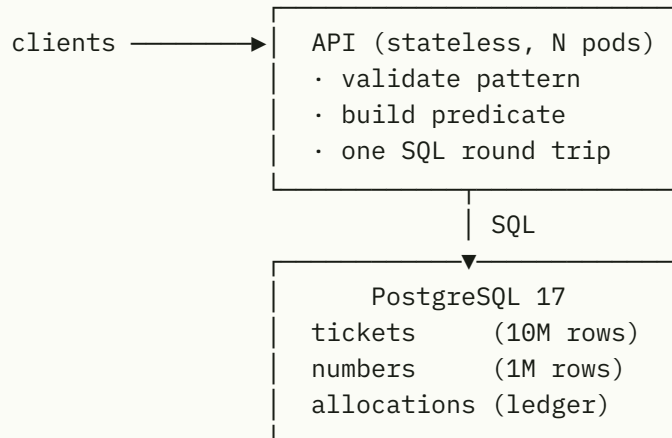
DELIVERABLE	DECISION	JUSTIFICATION	DETAIL
Solution architecture, data structures, algorithms	One stateless API in front of one PostgreSQL 17 instance holding tickets (10M rows), a static numbers table (10^6 rows, one per possible value, indexed per digit)	The search space is the 10^6 possible numbers, not the 10^7 tickets; it is fixed regardless of ticket count and stays in cache.	Sections 5.1 to 5.3

DELIVERABLE	DECISION	JUSTIFICATION	DETAIL
	and an append-only <code>allocations</code> ledger. Search resolves the pattern on <code>numbers</code> — fixed prefix → primary-key range, exact pattern → point lookup, otherwise per-digit B-trees — then joins into <code>tickets</code> through a partial index on available rows.	Total index footprint 201 MB, half of the per-digit alternative.	
Production database / storage	PostgreSQL 17, single instance. No cache, lock service or search engine.	<code>FOR UPDATE SKIP LOCKED</code> is a native, atomic, deadlock-free distribution primitive, so the hardest requirement needs no additional component. Reservation and audit share one transaction. Partial and expression indexes with a cost-based planner let the 10 ⁶ space be indexed several ways. One component to operate. Projected 36 % of a 6-core allocation at the illustrative 10,000 req/s target.	Sections 6.1, 6.4
Performance analysis	Strategy C: p95 46.7 ms at 6,068 req/s on one 500m-CPU pod, flat across all wildcard classes; four orders of magnitude faster than <code>LIKE</code> on the exact pattern. The saturated resource is the stateless API, which scales out. Accepted tradeoffs: one extra join (under 1 ms), no <code>ORDER BY</code> on search, allocation latency degrades as a pattern nears exhaustion.	Four strategies were measured on the same 10M rows. C is the only one that meets the latency target and the only one that moves the bottleneck out of the database.	Sections 6.2, 7.1, 7.2
Concurrency / distribution strategy	One SQL statement: a CTE selects candidates with <code>FOR UPDATE SKIP LOCKED</code> , updates them to reserved, and inserts the ledger row. Candidates are chosen by a random concrete number inside the pattern, with a pattern scan as fallback. Reservations expire through a TTL sweep.	A locked row is invisible to every other allocator until commit, nobody waits, and deadlock is impossible. Proven by 342,412 allocations under 200-way contention on one pattern with 0 duplicates and 0 ledger/state mismatches.	Sections 5.4, 8.1

The design is one stateless API, one PostgreSQL 17 instance, three tables. Nothing else is deployed: no cache, no lock service, no search engine. Every figure in this document was measured on PostgreSQL 17.11 (Section 6.4).

Architecture, data model, search, and concurrency

5.1 Architecture and API



Nothing else is deployed. A component that does not exist cannot fail, cannot drift from the database, and needs no operation. The database's row locking provides the concurrency guarantee (Section 5.4), which removes the usual case for Redis. The API holds no state, so throughput scales by adding replicas (Section 7.2).

ENDPOINT	PURPOSE
POST /v1/search	Preview matches, reserves nothing. Result limit capped at 100.
POST /v1/allocate	Atomically reserve matching tickets for a user. Limit capped at 20.
POST /v1/release	Return reserved tickets to the pool.
GET /healthz	Liveness. Process only; never touches the database (Section 7.3).
GET /readyz	Readiness. Pings the database.

A pattern must match `^[0-9*]{6}$` and is rejected at the handler before any database work; this rule is what allows the predicate builder to inline digits into SQL safely.

Result counts are *estimated* as $10^{\text{wildcards}} \times \text{tickets-per-number}$: a real `COUNT(*)` for `**3**` would scan a million rows for a figure that needs no precision.

5.2 Data model

```
CREATE TABLE tickets (  
  id          bigint    PRIMARY KEY,  
  number      int       NOT NULL,          -- 0..999999  
  status      smallint  NOT NULL DEFAULT 0, -- 0=available, 1=reserved, 2=sold  
  reserved_by int,  
  reserved_at timestamptz  
);  
  
CREATE TABLE numbers (                -- the 1M-row search space  
  n int PRIMARY KEY,                  -- 0..999999  
  d1 smallint NOT NULL, d2 smallint NOT NULL, d3 smallint NOT NULL,  
  d4 smallint NOT NULL, d5 smallint NOT NULL, d6 smallint NOT NULL  
);  
  
CREATE TABLE allocations (                -- append-only ledger  
  ticket_id bigint    NOT NULL,  
  user_id   int       NOT NULL,  
  at        timestamptz NOT NULL DEFAULT now()  
);
```

DECISION	REASON
<code>number</code> is <code>int</code> , not <code>char(6)</code>	Saves about 60 MB at 10M rows, compares faster, and digits are extracted arithmetically. Zero-padding is applied on output.
Digits are not stored on <code>tickets</code>	Six generated columns would add about 120 MB to the heap. The digit columns live on the 1M-row table at a fraction of that cost.
<code>status</code> is <code>smallint</code>	Smaller and faster to compare than an enum or text; sufficient for three states.
A separate <code>allocations</code> ledger	Makes the duplicate-distribution guarantee auditable with one query rather than inferred (Sections 5.4, 8.1).

`numbers` is static: one million rows generated once and never written again. It is the reason search cost does not grow with ticket count.

5.3 Search algorithm and indexes

For a pattern such as `1*3**6`:

1. **Validate** the pattern and split it into fixed digits and wildcards.

2. **Resolve the number space** on `numbers` . Two cases collapse to a range or a point: a fixed *leading* run (`123***`) becomes the primary-key range `n BETWEEN 123000 AND 123999` ; a fully specified pattern (`123456`) becomes `number = 123456` directly on `tickets` . Everything else uses per-digit B-trees.
3. **Fetch tickets** by joining those numbers into `tickets(number)` through a partial index restricted to `status = 0` .
4. **Stop early**. The `LIMIT` is pushed into the scan.

```
-- 123*** → a primary-key range on the 1M-row table
SELECT id, number FROM tickets
WHERE status = 0
      AND number IN (SELECT n FROM numbers WHERE n BETWEEN 123000 AND 123999)
LIMIT 20;

-- ****23 → per-digit indexes on the 1M-row table
SELECT id, number FROM tickets
WHERE status = 0
      AND number IN (SELECT n FROM numbers WHERE d5 = 2 AND d6 = 3)
LIMIT 20;
```

Seven indexes — six digit indexes on `numbers` at about 6.8 MB each, plus a partial B-tree on `tickets(number)` — **201 MB** at 10M tickets. The `numbers` share is fixed; only the ticket index grows, and as a partial index on `status = 0` it *shrinks* as inventory is sold.

TWO RULES DERIVED FROM MEASUREMENT

No ORDER BY on search. `ORDER BY id LIMIT 20` was the largest single cost in the system: at 10M rows the planner walks the primary key and probes the number table once per row through a Memoize node — 13.4 ms for `123***` against 0.148 ms without the ordering (90×), and 54–92× across the other narrow classes. Search results have no meaningful order, so the clause was removed and per-query time fell below a millisecond.

Never materialize the full match set. `**3***` matches about a million tickets; the user wants twenty. The `LIMIT` must reach the index scan so that time-to-first-page is constant rather than proportional to the match count.

5.4 Concurrency and distribution

The database alone meets this requirement.

```
WITH picked AS (  
  SELECT id FROM tickets  
  WHERE status = 0 AND <pattern predicate>  
  LIMIT $2  
  FOR UPDATE SKIP LOCKED           -- the guarantee  
) , upd AS (  
  UPDATE tickets t SET status = 1, reserved_by = $1, reserved_at = now()  
  FROM picked WHERE t.id = picked.id  
  RETURNING t.id, t.number  
) , ledger AS (  
  INSERT INTO allocations (ticket_id, user_id) SELECT id, $1 FROM upd  
)  
SELECT id, number FROM upd;
```

One statement is one transaction: selecting, reserving and recording either all happen or none do. `FOR UPDATE` locks the chosen rows; `SKIP LOCKED` makes a concurrent transaction **step over** rows another transaction holds and take the next available ones. Four properties follow: no two users can receive the same ticket, because a locked row is invisible to every other allocator until commit; no lock contention, so latency does not degrade as concurrency rises; no deadlock, because no transaction waits on another; and no external coordination.

Candidate selection

Taking candidates in `id` order creates a hot head: every reserved row stays at the front of the order, and each allocation walks past all of them. After 713 reservations a single allocate scanned 68,453 rows in 145 ms, and the cost grew with every call. The allocator therefore picks a **random concrete number inside the pattern** (`****23` → e.g. `481723`), performs a point lookup, and retries up to three times before falling back to a pattern scan. Concurrent users spread across $10^{\text{wildcards}}$ numbers, so collisions are rare and cost is constant regardless of inventory consumed. `SKIP LOCKED` provides correctness on both paths; the probe is a performance measure only.

Reservation expiry

A background sweep releases reservations older than five minutes. In production it runs as a scheduled job, once per interval regardless of API replica count.

Measured proof: 342,412 allocations under contention with zero duplicates (Section 8.1). Recorded as ADR-003 and ADR-004.

06 ALTERNATIVES CONSIDERED

What else was measured, and why it was rejected

6.1 Storage and coordination

Chosen: PostgreSQL 17, single instance.

REQUIREMENT	WHY POSTGRESQL MEETS IT
Duplicate-free distribution	<code>FOR UPDATE SKIP LOCKED</code> is a native, atomic, deadlock-free work-distribution primitive. This is the deciding factor: the hardest requirement in the challenge is a one-line feature of the database.
Wildcard search performance	Expression indexes, partial indexes and a cost-based planner let the 1M-number search space be indexed several ways. Measured p95 of 47 ms at 10M rows under 100 concurrent users.
Transactional integrity	Reserving a ticket and writing the audit ledger are one transaction. A separate lock service cannot offer this.
Operational simplicity	One component to deploy, back up, monitor and reason about. Total storage for 10M tickets and all indexes is about 1.4 GB.
Scaling headroom	The measured cost model (Section 7.2) projects a single instance at 36 % of a six-core allocation at the illustrative 10,000 req/s target.

Rejected

OPTION	WHY NOT
Redis as a cache in front	Adds a component and a failure mode, and a consistency problem where correctness matters most. Allocation must write to the database regardless, so the cache cannot own the decision.
Redis distributed lock	Requires lock expiry, recovery from a holder that dies, and still a database write. <code>SKIP LOCKED</code> provides the same guarantee transactionally at no cost.
<code>SELECT FOR UPDATE</code> without <code>SKIP LOCKED</code>	Correct but serializing: every allocator queues behind the first, and latency reaches seconds under contention.

OPTION	WHY NOT
Optimistic locking with a version column	Under contention on a popular pattern the retry rate climbs until throughput collapses.
Application-level in-memory lock	Cannot work across replicas, and API replication is the primary scaling path (Section 7.2).
Elasticsearch or a search engine	Solves text search, not a fixed six-digit key space, and offers nothing for atomic allocation, which is the actual difficulty.
Precomputing every pattern's results	10 ⁶ possible patterns with result sets up to a million rows. Storage and invalidation cost exceed the query cost avoided.

Recorded as ADR-001 and ADR-003.

6.2 Index strategies

Four strategies were implemented and measured on the same 10M-row dataset. A single strategy without alternatives would show only that it works, not that it is the right choice.

STRATEGY	WHAT IT DOES	INDEX SIZE
O Baseline	<code>lpad(number::text,6,'0') LIKE '____23'</code>	none usable
A Expression indexes	One partial index per digit on <code>tickets</code> , relying on the planner to combine them	405 MB
B Dimension table	Resolve the pattern on <code>numbers</code> , then join	201 MB
C B + prefix range	As B, but a fixed leading run becomes a key range and an exact pattern a point lookup	201 MB

Single-query cost (milliseconds, PostgreSQL 17.11, warm cache)

PATTERN	WILDCARDS	O	A	B	C
<code>123456</code>	0	829	130	5.1	0.087
<code>123***</code>	3	2.5	21.6	1.7	0.148
<code>12*4**</code>	3, scattered	1.5	21.6	1.3	0.158
<code>****23</code>	4	0.17	0.11	0.22	0.132

PATTERN	WILDCARDS	O	A	B	C
3*	5	0.05	0.05	0.33	0.206
*****	6	0.04	0.02	0.02	0.024
The baseline is competitive on the lower rows only because <code>LIMIT 20</code> stops a scan early when matches are abundant. The exact-match row, about ten matching tickets in ten million, is the decisive test: the chosen design is four orders of magnitude faster (about 9,500× on this pass; single-run figures vary between passes, so the order of magnitude is the defensible claim).					

Under load: 10 → 100 concurrent users, one 500m-CPU API pod

STRATEGY	P95 W0	P95 W3	P95 W4	P95 W5	P95 ALL	REQ/S	FAILED
C	46.8	46.9	46.5	46.7	46.7	6,068	0 %
B	176.0	114.1	103.3	103.6	108.3	975	0 %
A	5,663	4,503	4,503	4,392	4,705	25	0 %
O	1,320	1,049	1,018	1,053	1,051	128	94.3 %
Latency is flat across wildcard classes under Strategy C, the property Section 2.1 requires.							

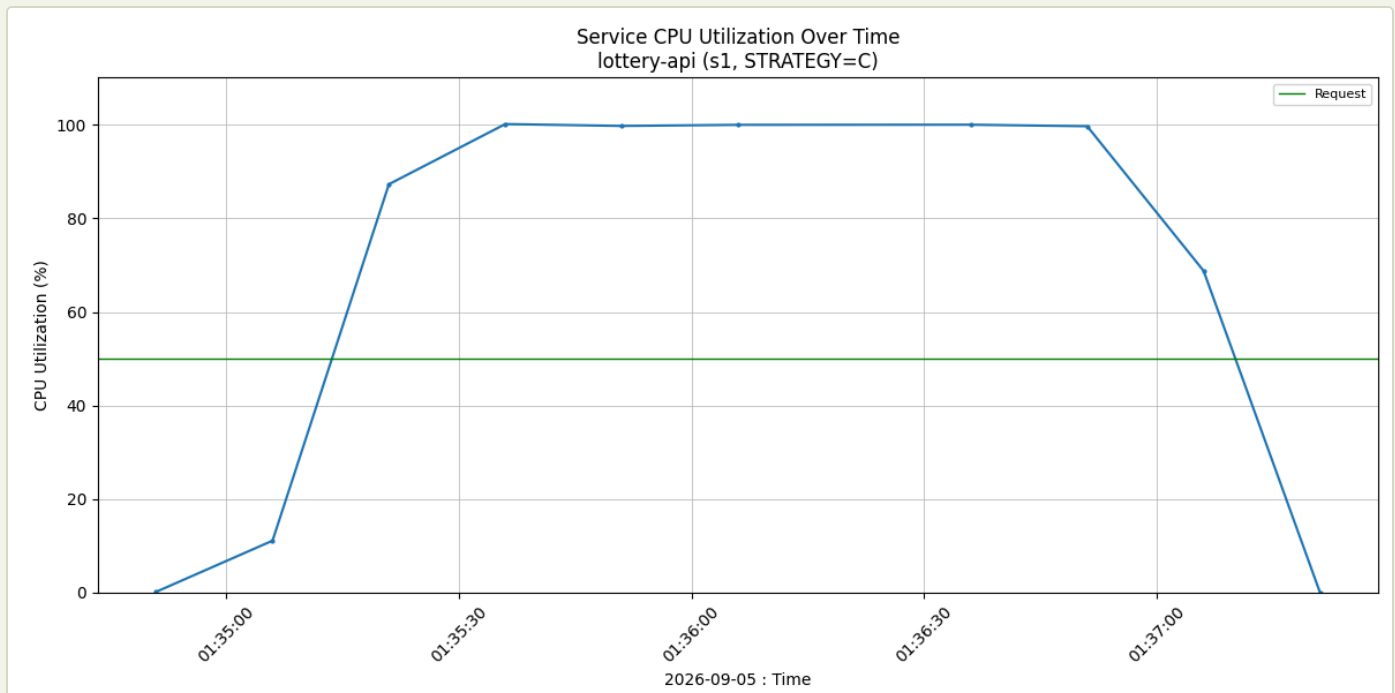
Where each strategy spends its resources

STRATEGY	API CPU	POSTGRES CPU	POSTGRES MEMORY FREE
C	100 % – at limit	85 %	48 MiB
B	24 %	102 % – at limit	17 MiB
A	2 %	102 % – at limit	25 MiB
O	1 %	101 % – at limit	235 MiB

This table decides the design. **Strategy C is the only strategy that moves the bottleneck out of the database.** Postgres retains headroom, so throughput scales by adding stateless API replicas. Under A and B the database is the constraint, and API scaling cannot help.

Postgres memory runs near its 1 GiB limit under every indexed strategy (working set 976–1,007 MiB for A, B and C): the hot set of a 10M-row table plus its indexes is about

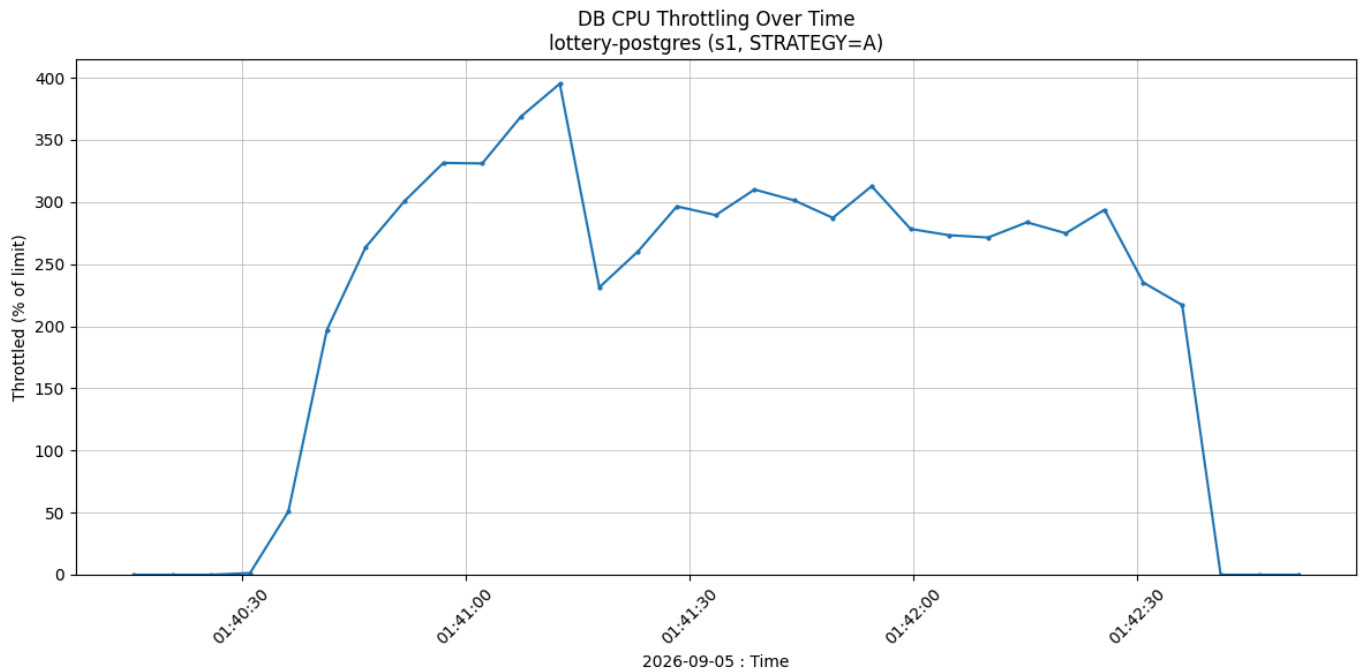
the size of the pod. Nothing was OOM-killed, but the baseline profile is tight, which is one reason Section 7.2 allocates 3Gi.



Under Strategy C the API pod saturates first. The constrained resource is the one that can be replicated. The green line is the pod's CPU request.

Why Strategy A fails

Strategy A indexes every digit and relies on the planner to intersect them. A single digit is 10 % selective, so each index still identifies a million rows out of ten million; combining six costs more than it saves, and the planner uses one index and filters with the rest.



Strategy A had the database denied up to 395 % of its CPU allocation per second, and 405 MB of indexes delivered 25 req/s. Index size and index *selectivity* are different properties: an index that does not eliminate most rows costs more than a sequential scan.

Recorded as ADR-002.

6.3 Strategy D — measured and rejected

PostgreSQL 17 satisfies `= ANY(array)` in a single B-tree descent, which makes a further query shape viable: the API enumerates the matching numbers and sends them as an `int[]` instead of a subquery against `numbers`. Strategy D applies to patterns with at most three wildcards (1,000 values, about 4 KB); wider patterns fall back to C.

Measured with classes `w0` (exact), `w3lead` (`ddd***`) and `w3scatter` (`dd*d**`). The last has no leading run, so C has no key range to exploit; it is the only class where D can differ materially (`d1-C/`, `d1-D/`):

STRATEGY	P95 ALL	MAX	THROUGHPUT	API CPU	DB CPU
C	48.4 ms	153 ms	4,856 req/s	100 %	71 %
D	55.7 ms	77 ms	6,110 req/s	100 %	35 %

Both runs: 0 HTTP errors, 0 duplicate tickets, 0 ledger mismatches. Per-class p95 is flat within 0.3 ms for both strategies (C: 48.4 / 48.4 / 48.6 ms; D: 55.5 / 55.7 / 55.7 ms).

D moves work from the database to the API: peak database CPU falls from 71 % to 35 %, throughput rises 26 %, p95 rises 15 %, and the API is the saturated resource in both runs. The single-query plans in `docs/explain-k8s-10M.txt` agree: D is marginally faster on three-wildcard patterns and indistinguishable elsewhere. Two conclusions follow. **D produces database headroom, not user-visible speed:** the saving comes from dropping the join against `numbers`, while the saturated API pod is what the user waits on. And **throughput and latency moved in opposite directions:** more requests pass through the same pod, but the tail did not improve.

Strategy D is not adopted. Section 7.2 places the database at 36 % of its allocation at the illustrative 10,000 req/s target, so database headroom solves a problem this system does not have, while D adds a second predicate path, a per-request allocation of up to 1,000 integers, and 4 KB of extra traffic per narrow query. It becomes the right choice only if the database becomes the constraint, at which point this measurement indicates it would halve database CPU. The implementation remains in the proof of concept behind `STRATEGY=D`. Recorded as ADR-008.

6.4 PostgreSQL version: 17 throughout

Every figure in this document was taken on **PostgreSQL 17.11** (`postgres:17-alpine`), on one 10M-row dataset, one baseline pod profile (one 500m API pod, database at 2000m/1Gi) and one set of capture scripts. Each evidence bundle records `pg_version=17.11` in its `41-run.txt`; no figure is restated or scaled from another version.

Deploy on 17. It is the measured version, and the design depends on nothing outside it: the duplicate-free guarantee rests on `SKIP LOCKED`, the search path on partial B-tree indexes and a primary-key range, all of which behave identically on 16. The one 17-specific capability examined, the single-descent `= ANY(array)` scan, motivated Strategy D, which Section 6.3 declines. A later major-version upgrade is a version bump with no schema or SQL change; the load-test suite should be re-run rather than

the figures assumed to carry over, because a single-query benchmark and a concurrency benchmark answer different questions. Recorded as ADR-007.

07 CROSS-CUTTING CONCERNS

Performance tradeoffs, capacity, operations, observability

7.1 Performance tradeoffs accepted

- **One extra join.** Strategy C reads two tables. Measured cost: fractions of a millisecond, because the number table remains in cache.
- **One extra table to seed.** One statement, run once.
- **Leading wildcards bypass the range optimization.** `****23` falls back to per-digit indexes at 0.13 ms; the fallback is not a weak path.
- **Allocation degrades near exhaustion.** With `****23` 96 % drained, p95 rose from 75 ms to 510 ms as random probes increasingly missed. Withdrawing a nearly sold-out pattern is a product decision, not a database problem, but the behaviour must be designed for.

7.2 Capacity and scaling

ILLUSTRATIVE TARGET

The 10,000 requests/sec target and the 60 % utilisation ceiling are values chosen by the author to demonstrate the sizing method; the challenge specifies neither. The model below applies unchanged to any other target.

Because cost per request is stable, capacity is planned arithmetically. A stepped load test drove 1,000 / 2,000 / 3,000 / 4,000 requests per second for 60 s each on the proposed profile of four 500m API pods and a 6-core database, sampling only the steady 50 s of each step:

DRIVEN RATE	API CPU (4 PODS)	API % OF LIMIT	POSTGRES CPU	POSTGRES % OF LIMIT	POSTG
1,000	100m	5.0 %	320m	5.3 %	
2,000	192m	9.6 %	484m	8.1 %	
3,000	293m	14.6 %	706m	11.8 %	
4,000	415m	20.7 %	942m	15.7 %	
k6 delivered 579,999 of 580,000 requested calls (100.00 %) with zero failures, zero dropped iterations and p95 1.12 constant step increments license a linear model.					

CPU used = background + cost per request × rate

API

=

-4.6 m-core

+

0.1049 m-core per req/s

→

1,044m at 10,000 req/s

Postgres

=

112.1 m-core

+

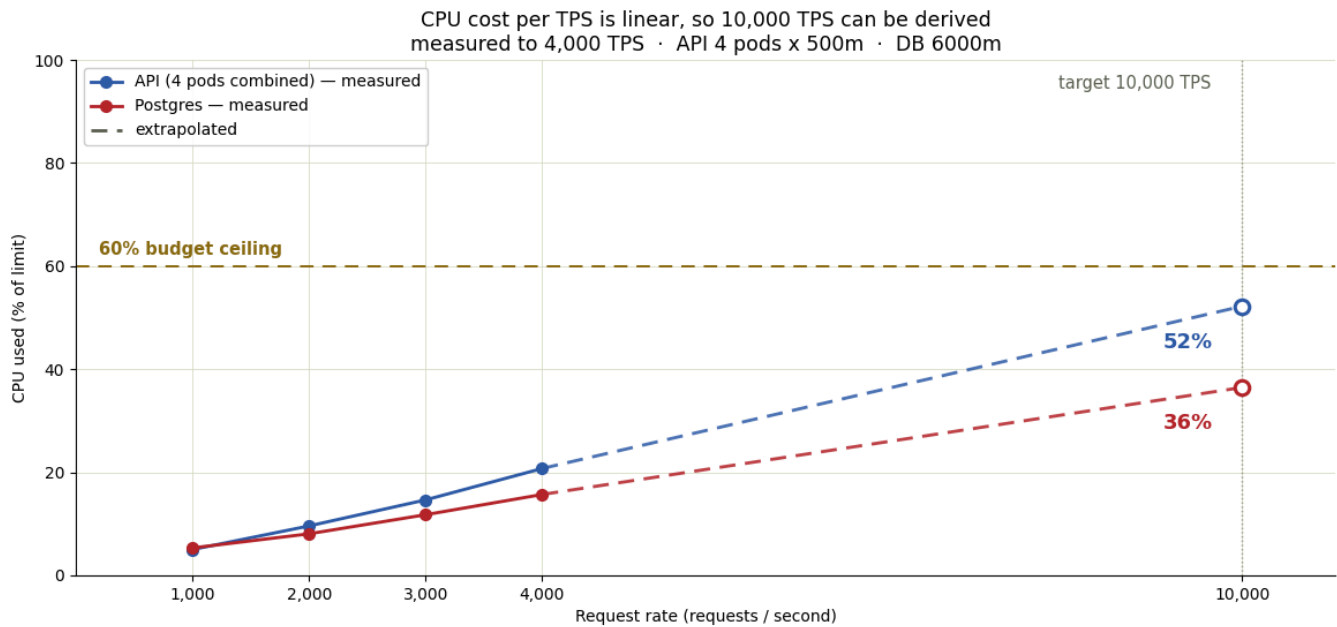
0.2075 m-core per req/s

→

2,187m at 10,000 req/s

Fitting the first and last step separates fixed background work from marginal per-request cost. The slightly negative API intercept is an artefact of a two-point fit; the API has no material fixed cost, and almost all of its CPU scales with load. At the 10,000 requests/sec target, holding both components under 60 % of their limits:

COMPONENT	REPLICAS	CPU LIMIT	MEMORY LIMIT	PROJECTED AT 10,000 REQ/S
API stateless — scale out	4	500m per pod	128Mi per pod	52 % CPU · 13 % memory
PostgreSQL single instance	1	6,000m	3Gi	36 % CPU · 44 % memory



Solid lines are measured, dashed are derived. The horizontal line is the 60 % budget. At 10,000 requests/sec both components remain below it. Plotting percentage-of-limit rather than raw CPU lets two differently sized components share one budget line.

Three points a CPU-only model would miss. **Database memory does not grow linearly with load:** it grows with the hot data pulled into cache and then flattens, with per-step increments of 232 → 174 → 41 MiB converging near 1,363 MiB. A linear extrapolation would have called for about 2,217 MiB; at the 2,560Mi limit actually tested the plateau is 53 %, inside the budget with no margin, so 3Gi (44 %) is recommended. **Connection pooling is not a constraint:** required concurrency is rate × service time = 10,000 × 0.69 ms ≈ 7 connections against 40 provisioned. **Scale the API out, not up:** the Go runtime is pinned to one processor per pod, so a larger pod does not help; more pods do, and they add fault tolerance.

Beyond about 20,000 requests/sec the single database becomes the limit. The next step is read replicas for search only; allocation stays on the primary because `SKIP LOCKED` requires the authoritative copy.

7.3 Reliability and operations

Liveness must not depend on the database. Under the baseline strategy the connection pool filled with slow queries, the readiness probe timed out, Kubernetes

removed the pod from the Service, and 15,592 of 16,528 requests failed. That is correct behaviour. Had the *liveness* probe also queried the database, Kubernetes would have restarted healthy pods during a database slowdown and turned a degradation into an outage. `/healthz` therefore checks the process only; `/readyz` pings the database. Recorded as ADR-006.

Resource limits must reach the runtime. A Go process reads the host CPU count, not its cgroup quota. `GOMAXPROCS` and `GOMEMLIMIT` are set explicitly; without them the runtime over-allocates threads and is eventually OOM-killed.

Stability. A 10-minute soak of mixed search, allocate and release traffic: 942,015 requests, zero errors, zero restarts, no OOM kills, API memory peaking at 26 MiB of a 128 MiB limit.

7.4 Observability

Signals required in production: query plans via `pg_stat_statements` ; lock contention via `pg_locks` and `pg_stat_activity` ; per-pod CPU and memory against limits; and CPU throttling, which average-CPU graphs hide and which was the mechanism behind Strategy A's collapse (Section 6.2). A full observability stack is a non-goal (Section 3.2).

08 VALIDATION AND EVIDENCE

Every claim, and where to verify it

8.1 Load-test evidence for duplicate-free distribution

Two contention runs, each with 200 concurrent users on a single pattern for 60 seconds.

	****23 ~100K TICKETS	**3*** ~1M TICKETS
Allocations in 60 s	95,630	246,782
Ledger rows = reserved tickets = distinct ids	95,630	246,782
Tickets issued twice	0	0

****23 ~100K TICKETS

3 ~1M TICKETS

Ledger rows disagreeing with ticket state

0

0

HTTP errors

0

0

342,412 allocations, zero duplicates. The audit is one query that must return no rows: `SELECT ticket_id, count(*) FROM allocations GROUP BY ticket_id HAVING count(*) > 1;`

8.2 Automated tests

The same invariant is asserted in `go test` against a real PostgreSQL: 50 goroutines compete for 120 tickets, every ticket is issued exactly once, and an exhausted pool returns an empty result rather than an error. The suite runs against every index strategy, including the rejected ones, so correctness is independent of the search path.

8.3 Evidence index

CLAIM	SOURCE
Query plans and index sizes	<code>docs/explain-k8s-10M.txt</code>
Cost of the rejected <code>ORDER BY</code> design	<code>docs/explain-k8s-10M-orderby.txt</code>
Strategy comparison under load	<code>docs/evidence/s1-C/</code> , <code>s1-B/</code> , <code>s1-A/</code> , <code>s1-0/</code>
Zero duplicate distribution	<code>docs/evidence/s2/30-verify.txt</code> , <code>s2b/30-verify.txt</code>
Stability over 10 minutes	<code>docs/evidence/s3/</code>
Capacity model for the illustrative 10,000 req/s target	<code>docs/evidence/c1-capacity/50-capacity.json</code>
Strategy D against Strategy C	<code>docs/evidence/d1-C/</code> , <code>d1-D/</code>
PostgreSQL version of every run	<code>pg_version=17.11</code> in each bundle's <code>41-run.txt</code>
Full analysis of every run	<code>docs/RESULTS.md</code>

Each bundle contains the complete load-generator log, the summary as JSON and HTML, resource charts, the raw samples the charts were drawn from, and the SQL verification output.

What this design has not yet proven

- **10,000 req/s is calculated, not observed.** The highest measured rate is 4,000 req/s; the illustrative target is a $2.5\times$ linear extrapolation. Four data points support linearity but do not prove it further out. Validate with a load generator on separate hardware before committing.
- **Near-exhaustion behaviour is only partly characterized.** Allocation latency degrades when a pattern is nearly sold out; the threshold at which a pattern should be withdrawn from sale has not been determined.
- **All measurements come from a single node,** where API, database and load generator share one CPU pool. Absolute throughput on separate hardware would differ; the comparison between strategies, on which the recommendation rests, would not.
- **Failure modes are untested.** No measurement covers pod loss, rolling updates or database failover. Capacity is reduced during those events, so provision at least one replica beyond the computed minimum.
- **A skewed inventory distribution is untested.** All measurements start from a fully available pool. With 80 % of tickets sold the partial index shrinks but the random-probe hit rate falls; both effects need measuring.
- **Out of scope by design:** authentication, payment, draw scheduling, replication and any caching layer (Section 3.2).

Decision records, glossary, and the scoring map

A. Architecture decision records

One record per decision, in `adr/`. Each states the context, the decision, the alternatives rejected, the consequences accepted, and the evidence that supports it.

ADR	DECISION	SECTION
ADR-001	Use a single PostgreSQL instance; no cache, no lock service	6.1
ADR-002	Resolve patterns on a 1M-row <code>numbers</code> table, then join (Strategy C)	5.3, 6.2
ADR-003	Guarantee duplicate-free allocation with <code>FOR UPDATE SKIP LOCKED</code> in one statement	5.4
ADR-004	Allocate by random concrete-number probe before falling back to a pattern scan	5.4
ADR-005	No <code>ORDER BY</code> on search	5.3
ADR-006	Liveness must not touch the database; readiness must	7.3
ADR-007	Deploy on PostgreSQL 17	6.4
ADR-008	Do not adopt Strategy D	6.3

B. Glossary

TERM	MEANING IN THIS DOCUMENT
p95	The latency that 95 % of requests were faster than. Used instead of the mean because the mean hides the slow tail users experience.
VU	Virtual user: one simulated client in the load generator, issuing requests in a loop. 200 VUs means 200 concurrent clients.
wildcard class	Patterns grouped by how many <code>*</code> they contain (w0, w3, w4, w5). Latency is reported per class because result-set size differs by six orders of magnitude between them.
SKIP LOCKED	Option on <code>SELECT ... FOR UPDATE</code> that makes a transaction skip rows another transaction holds instead of waiting for them. The basis of the duplicate-free guarantee.
BitmapAnd	The planner's method of intersecting several indexes. Pays off only when each index is highly selective; a single-digit index is 10 % selective, which is why Strategy A fails.
partial index	An index restricted by a <code>WHERE</code> clause, here <code>WHERE status = 0</code> , so it covers only available tickets and shrinks as inventory sells.
working set	Memory in use excluding reclaimable page cache. Raw usage of a PostgreSQL container sits near 100 % of its limit because it fills the cgroup with cache; the working set is the meaningful figure.

TERM	MEANING IN THIS DOCUMENT
CPU throttling	Time a container was denied CPU because it exceeded its quota. Invisible in average CPU graphs; measured directly from the cgroup.
liveness / readiness	Liveness asks whether the process is alive; failing it restarts the pod. Readiness asks whether the pod can take traffic; failing it removes the pod from the load balancer without restarting it.

C. Mapping to the challenge's evaluation criteria

CRITERION	WHERE IT IS ADDRESSED
Feasibility	Section 5.1 architecture, Section 5.2 data model, Section 5.3 algorithm; a working implementation was built and measured
Performance	Section 6.2 four-way comparison at 10M rows, Section 7.2 capacity model
Correctness	Section 5.4 concurrency design and the 342,412-allocation audit in Section 8.1
Real-world practicality	Section 6.1 database choice and rejected alternatives, Section 7.3 operations, Section 9 limitations
Creativity	Section 2.1 the one-million search space, Section 5.3 collapsing a fixed prefix to a key range, Section 5.4 the random-probe allocation path

Design document for the Lottery Search System challenge. The markdown source is `DESIGN_DOCUMENT.md`; supporting analysis is in `docs/RESULTS.md` and the capacity plan, and all raw measurements are under `docs/evidence/`.